Data Science for Political Campaigns

Week 10 — Turnout models

Prof. Joshua Kalla

November 11, 2026

Roadmap

- **Last week:** four GOTV tactics, cost per vote, and updating a belief with new evidence.
- **This week:** turnout models. How to build one and how to test it.
- **Next week:** forecasts and calibration.

This Week's Case

Case: You are the field director of a state senate campaign

The seat decides control of the chamber. There are 60,000 registered voters in the district and your volunteers can knock about 9,000 doors for a get-out-the-vote program before election day.

The campaign paid Keystone Analytics $9,000 for a turnout file. Walk lists go to the printer Friday.

What Keystone Sent

One row per registered voter and a column called `keystone_score`.

Voter	Age	Voted 12 / 14 / 16 / 18	`keystone_score`
Sonia Vargas (906218)	40	no / yes / yes / yes	0.94
Hannah Weiss (957674)	48	yes / no / yes / no	0.73

From the methodology sheet: estimated on prior general election participation together with age. Validated against the 2024 general election, the model classified 80.0% of registered voters correctly.

You have the same voter file they used.

Discuss

- Keystone says the model is right 80.0% of the time. What would you need to know to decide whether that is good?
- Rachel wants to canvass the 9,000 voters with the highest turnout scores. Marcus wants the 9,000 lowest. Katie wants 9,000 with scores near 0.5. Who should you canvass?

Pair up. Three minutes!

What a Turnout Model Is

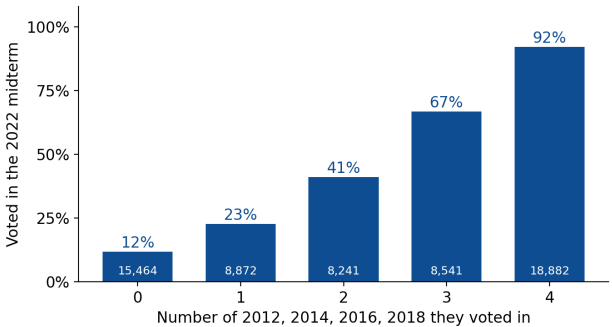
Framework

A turnout model predicts, for each registered voter, the probability that they vote in an upcoming election. That probability is their propensity.

Three steps:

1. **Pick a past election where you know who voted.** Ours is 2022. Your race is a 2026 midterm, and 2022 is the last midterm.
2. **Find the pattern that predicts who voted, from what you knew beforehand:** whether they voted in 2012, 2014, 2016 and 2018, and their age. That is a regression.
3. **Apply the pattern to everyone** to get a score for 2026.

Past Voting Predicts Future Voting



All 60,000 registered voters, grouped by how many of the four generals from 2012 to 2018 they voted in.

Voted in none of them: 11.9% voted in 2022.

Voted in all four: 92.1%.

Step 2: Find the Pattern

```
smf.ols('voted_2022 ~ voted_2012 + voted_2014 + voted_2016 +
voted_2018 + age')
```

Variable	Coefficient	In points
Intercept	0.1449	starting point
voted_2012	0.1041	+10.4
voted_2014	0.2767	+27.7
voted_2016	0.1180	+11.8
voted_2018	0.3306	+33.1
age	−0.0010	−0.1 per year
$N = 42{,}000$	$R^2 = 0.449$, the share of the variation the model accounts for	

Multiply a coefficient by 100 to read it in percentage points, same as Week 2. Add up the rows that apply to a person and you have their score.

We built it on 42,000 of the 60,000 voters. The other 18,000 are held back to test it.

What the Coefficients Say

- 2014 and 2018 were midterms. 2012 and 2016 were presidentials. Voting in a past midterm is worth about 30 points. Voting in a past presidential is worth about 11.
- Age is worth -0.1 points a year. From age 20 to 90, that is 7 points, against 33 for one midterm.

Why is age so small? Older people have voted more often, so vote history already accounts for most of it.

A Linear Probability Model

Framework

A linear probability model is a regression with a 0/1 outcome. Each coefficient is a change in the probability, so you can read it in points.

A straight line can go below 0 or above 1. Fit this model to 2024 and 16% of voters get a score above 1.00.

The fix: trim scores at 0 and 1. Keystone did. Logistic regression is the fancier fix (bounded between 0 and 1).

Step 3: Score Everyone for 2026

Same coefficients. Move every voter's history forward four years:

Coefficient	To find it (predict 2022)	To use it (predict 2026)
$+10.4$	voted in 2012	voted in 2016
$+27.7$	voted in 2014	voted in 2018
$+11.8$	voted in 2016	voted in 2020
$+33.1$	voted in 2018	voted in 2022
-0.1 per year	age	age $+4$

You cannot check a 2026 score until after the election. So we test the model on 2022 instead, where we know who voted, using the 18,000 voters we held back.

What About New Voters?

Not registered: they are not on the file, so the model cannot score them.

Newly registered: someone who just turned 18 has a 0 in every vote-history column. But a 0 means one of two things: *did not vote* or *could not vote*.

The model has seen this before. People who turned 18 in 2022 also had all 0s, and the model was fit to how they voted. If 18-year-olds in 2026 vote like 18-year-olds in 2022, their scores are right.

That works only if the model can tell a new voter's 0s from the 0s of someone who never votes. The fix is to tell the model which elections each person could have voted in. Most voter files have a registration date. Ours has age.

Keystone Says 80.0%. Is That Good?

Framework

Accuracy: call anyone above 0.5 a voter and anyone below a non-voter. Accuracy is the share the model got right.

Keystone tested its scores on 2024, the election it was fit to.

Before you judge 80.0%, compare it to a model that writes the more common answer on *every* row. Call its accuracy the free answer.

Accuracy Has a Floor

In 2024, 75.9% of these voters voted. "Everyone votes" is right 75.9% of the time.

	Accuracy	The free answer	Work done
Keystone, tested on 2024	80.0%	75.9%	4.1 points

A high accuracy number in a high-turnout election is mostly the turnout rate.

From this week's reading

Google, Machine Learning Crash Course: "For heavily imbalanced datasets, where one class appears very rarely, say 1% of the time, a model that predicts negative 100% of the time would score 99% on accuracy, despite being useless."

Confusion Matrix

Framework

A **confusion matrix** sorts every voter into four boxes: what the model said against what they did.

	They voted	**They did not**
Model said "will vote"	**true positive (TP)**	**false positive (FP)**
Model said "will not vote"	**false negative (FN)**	**true negative (TN)**

Positive means the model said yes. *True* means it was right.

Green: the model got them right. **Gold**: the model got them wrong.

Accuracy, Precision, and Recall

$$\text{accuracy} = \frac{TP + TN}{TP + FP + FN + TN}$$

Of everyone, the share the model got right.

$$\text{precision} = \frac{TP}{TP + FP}$$

Of the people the model called voters, the share who voted.

$$\text{recall} = \frac{TP}{TP + FN}$$

Of the people who voted, the share the model called voters.

Our Model's Numbers

Our model, tested on the 18,000 held-out voters in 2022:

	They voted	**They did not**
Model said "will vote"	**7,526**	**1,717**
Model said "will not vote"	**1,645**	**7,112**

accuracy $= (7{,}526 + 7{,}112) \,/\, 18{,}000 = 81.3\%$

precision $= 7{,}526 \,/\, (7{,}526 + 1{,}717) = 81.4\%$

recall $= 7{,}526 \,/\, (7{,}526 + 1{,}645) = 82.1\%$

What If You Assume Everyone Votes?

Compare our model to one that calls everyone a voter. In 2022, 50.9% of these voters voted.

	Our model	**Assume everyone votes**	**Why that number**
Accuracy	81.3%	50.9%	Right about the 50.9% who voted, wrong about everyone else
Precision	81.4%	50.9%	Everyone is called a voter, and 50.9% of them voted
Recall	82.1%	100%	Every voter was called a voter

Accuracy and precision are the same here only because everyone is called a voter. Calling more people voters always raises recall. It usually lowers precision.

Which Mistake Costs More?

The program	**False positive** said "will vote," did not	**False negative** said "will not," did	**Watch**
GOTV: knock everyone the model says will *not* vote	You skipped her and she stayed home. A potential vote lost. **This costs more:** she is who the program is for.	You knocked a sure voter. You lose $6.	Precision: it counts false positives
Persuasion mail to everyone the model says *will* vote	A mailer to someone who will not vote. You lose $0.60.	A voter you never mailed. A lost chance to persuade her. **This costs more:** a mailer is cheap.	Recall: it counts false negatives

Precision falls with every false positive. Recall falls with every false negative. Watch the one that counts your costlier mistake. Accuracy treats the two mistakes as equally bad.

Calibration

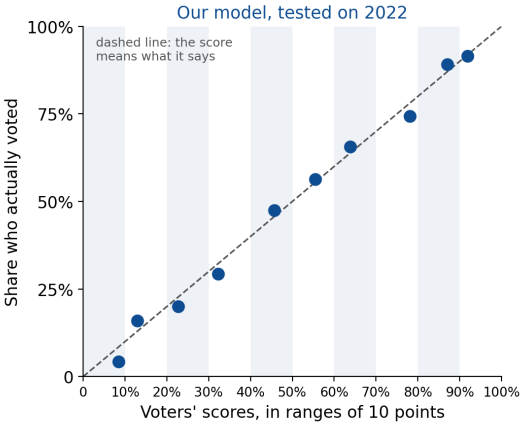
Framework

A model is calibrated when its numbers mean what they say: of the people it scores 0.60, about 60% vote.

Accuracy, precision and recall only use yes or no. Calibration uses the score itself.

To check it: group voters by score (0–10%, 10–20%, and so on). In each group, compare the average score to the share who voted.

Our Model, Checked



Each dot is everyone whose score fell in one shaded range. Up: the share who voted in 2022. Across: their average score.

So a dot sits at its voters' average, not the middle of the range. No one scores above 0.94, so the top range averages 0.92.

Every dot is close to the line.

The worst is the bottom range: the model said 8.5%, and 4.2% voted.

Overfitting

Framework

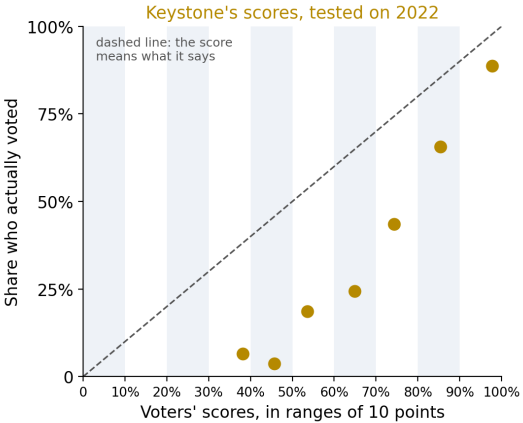
Overfitting: a model can fit the voters it was built on by memorizing them, and then predict new voters badly. The fix is to build on some voters and test on the rest.

What happens if you build on only 300 voters?

Built on 300 voters	**Accuracy on those 300**	**On 18,000 new voters**
Our five predictors	78.7%	81.3%
Same five, plus every combination	84.3%	76.7%

The complicated model memorized the 300. So when a vendor says "validated," ask: on which voters?

The Same Scores, On a Midterm



Keystone fit its model to 2024, a presidential year. Your race is a 2026 midterm. So test its scores on 2022.

More people vote in presidential years. Of the 18,000 voters we held back, 75.9% voted in 2024 and 50.9% in 2022.

Keystone's scores average 75.6%, close to 2024 turnout.

Every dot is below the line. The 1,905 people Keystone scored around 0.46 voted at 3.7%.

Accuracy falls from 80.0% to 61.4%, against a free answer of 50.9%.

Types of Voters

	Plans to vote	**Does not plan to vote**
Supports you	Make sure they vote	Make them likelier to vote
Undecided	Change their mind	Change their mind, then get them out
Supports the other	Leave alone	Leave alone

A turnout model tells you who will vote. It tells you nothing about who supports you.

The file has party registration, which is not the same as support.

Support Scores

Framework

A support score predicts, for each voter, the probability that they support your candidate.

You cannot build it the way we built the turnout model, because of the secret ballot (Week 4). So you run a survey:

1. Survey a random few thousand people on the file and ask who they support.
2. Regress that answer on what the file has for everybody: vote history, age, party.
3. Apply the coefficients to everyone you did not survey.

Two problems. It has every problem of the survey it was built from (Week 8), and you can only test it by running *another* survey.

From this week's reading

Pew: a voter file records *whether* you voted, never *who* you voted for. Pew checked five vendors' modeled race against what its survey panel said: 74% to 85% correct. One file identified only 56% of self-reported Black panelists.

What the Score Does Not Tell You

The score is the probability someone votes if you do nothing.

What you need is different: how much does your knock change that? That is a treatment effect. Nothing in the voter file contains it.

Experiments can tell you whether the treatment effect is larger among people with low, middle, or high turnout scores.

Takeaway

- A turnout model is a regression on the voter file: pick an election, find the pattern, apply it to 2026. Past voting is by far the strongest predictor.
- Compare accuracy to what you would get by calling everyone a voter.
- Precision and recall count different mistakes. Which one matters depends on which mistake costs you more.
- Test a model on voters it has never seen, and check calibration, not just accuracy.
- A model is fit to one kind of election. To predict a midterm, fit the model to a midterm.
- The score is the probability someone votes if you do nothing. It is not the effect of your knock.

Lying With Data This Week

Lying with data: An accuracy number quoted with no floor beside it. A model validated on the data it was built from. And a model that predicts *who votes*, sold as a model that predicts *who you can move*.

Read before Week 11 — Nov 18

Next week: forecasts and calibration. Broockman, Kalla, Caballero & Easton (2024): practitioners predicting which messages persuade. Tetlock, "How to Win at Forecasting." Silver, "When We Say 70 Percent, It Really Means 70 Percent." And FiveThirtyEight's calibration charts: look at two forecasts, one politics and one sports.

The Checklist So Far

1. **W1:** Conflating fixed and marginal costs.
2. **W2:** Presenting an observational comparison as a causal effect.
3. **W3:** Applying a result from one setting to a different one.
4. **W4:** Cherry-picking the winning arm from a multi-arm test.
5. **W5:** Treating an underpowered null as evidence of no effect.
6. **W7:** Reporting the complier comparison as a causal effect.
7. **W8:** Cherry-picking polls; house effects; ignoring nonresponse bias.
8. **W9:** Ranking tactics by effect size or cost per attempt instead of cost per vote.
9. **W10:** An accuracy number quoted with no floor beside it, a model graded on the data it was built from, and a model of *who votes* sold as a model of *who you can move*.

Break: 5 Minutes

$\sim$ 5 min $\sim$

Live Coding

Open `livecode.ipynb`. Data: `district_voter_file.csv`, 60,000 registered voters.

1. Test the file Keystone sent, starting with the accuracy floor.
2. Split train and test, fit the model, read the coefficients.
3. Calibration, twice: fit to a presidential, fit to a midterm.
4. Build the walk list.

Problem Set This Week

Open `problem_set.ipynb`.

1. Reproduce the accuracy floor for both elections.
2. Build the confusion matrix and compute precision and recall by hand.
3. Check calibration, and find a group of voters the model gets badly wrong.
4. A memo to Rachel: which 9,000 doors, and what would change your mind.

Recommendation first, then the reasons.